# Project 10

A Front Door for club_db | CIS 4720 Database Management and SQL

| Points | Time | Work in | Turn in |
|---|---|---|---|
| 10 | About 2 hours | project10_starter.sql | Your GitHub repository URL |

Homework. Work on your own. Textbook: Coronel and Morris, *Database Systems: Design, Implementation, and Management*, 14th edition, Chapter 5, Section 5.8.
The due date is listed on Canvas LMS.

## What you are building

In Project 9 you taught club_db to keep its own books: insert a payment, and triggers fill in the date, update the member's total and write the log. But anybody using the club's system still has to know the tables, the column names and which columns to leave empty.

In this project you build a **front door**: a set of stored procedures that are the only thing a program needs to know about. To take a payment it calls make_payment. It never touches the payment table directly, and it never needs to know the triggers exist.

| Procedure | What it does | Step | You write |
|---|---|---|---|
| list_members | Lists every member | 2 | All of it |
| member_history | Lists one member's payments | 3 | All of it |
| add_member | Adds a new member | 4 | The INSERT |
| make_payment | Takes a payment | 5 | The INSERT |
| refund_payment | Removes a payment | 6 | Most of it |
| list_members again | Changed to show the biggest payers first | 7 | All of it |

### This follows Labs 5f and 5g

Every piece is something you did in class on cis_students_db. The read only procedures are Lab 5f Steps 3 to 5. Changing a procedure is Lab 5f Step 6. The INSERT and DELETE procedures are Lab 5g Steps 3, 5 and 7. Keep your Lab 5f and 5g files open as a model.

Nothing in this project is brand new. The challenge is doing it without the class around you.

### Label and comment every step

Each step has a numbered heading already in project10_starter.sql. Put your statements under the right one, with a comment in your own words above every statement. It is worth a full point.

# Part 1. Set up, and two procedures that read

## Step 1. Create the schema and run the starter file

Make a folder on OneDrive called **CIS4720_Project10** and download **project10_starter.sql** into it. Open the folder in VS Code and connect SQLTools to **information_schema**. Run the three setup statements at the top of the file one at a time, then add a second connection with Database set to **club_db** and use that one.

Run the rest of the file one statement at a time. It rebuilds Project 9 finished, including the function and the three triggers, which you select as whole blocks. Then check:

| Check | What you should see |
|---|---|
| SELECT * FROM member; | Noor 45.00, Caleb 45.00, Sofia 60.00, Desmond 0.00 |
| SELECT * FROM payment; | Three payments: 5000, 5002 and 5003 |
| SELECT * FROM payment_log; | Five rows, four ADDED and one REMOVED |

Payments 5000 and 5002 carry the date you ran the starter, because the Project 9 trigger filled it in.

## Step 2. list_members

Yours from start to finish. Lab 5f Step 3 is the model.

| Requirement | Detail |
|---|---|
| Name | list_members, with no parameters |
| Columns | member_id, first_name, last_name, total_paid |
| Order | By last_name, A to Z |

Create it, then call it.

| member_id | first_name | last_name | total_paid |
|---|---|---|---|
| 103 | Desmond | Blake | 0.00 |
| 100 | Noor | Haddad | 45.00 |
| 102 | Sofia | Marchetti | 60.00 |
| 101 | Caleb | Ortiz | 45.00 |

## Step 3. member_history, with an IN parameter

Also yours from start to finish. Lab 5f Step 4 is the model.

| Requirement | Detail |
|---|---|
| Name | member_history |
| Parameter | One IN parameter, p_member_id, INT |
| Columns | payment_id, amount, paid_on, method, from the payment table |
| Rows | Only that member's payments |

Call it for member **100** and for member **103**.

| CALL for | What comes back |
|---|---|
| Member 100, Noor | One row: payment 5000, 45.00, the date you ran the starter, CASH |
| Member 103, Desmond | No rows. He has never paid |

## Part 2. Three procedures that change data

These three are where the front door earns its keep. Each holds one statement, and the triggers you built in Project 9 do everything else.

### Step 4. add_member

```
-- adds a new member, who joins today
CREATE PROCEDURE add_member(IN p_first VARCHAR(30),
                            IN p_last VARCHAR(30))
BEGIN
    -- store the new member, with today as the joining date
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<
END
```

**Writing the statement that goes in the gap**

An INSERT into member, like the one in Lab 5g Step 3, with one difference: one of the values does not come from a parameter.

**1. INSERT INTO member** and, in brackets, the three columns you are filling: first_name, last_name, joined_on.

**2. VALUES** and, in brackets, the two parameters in the same order, then **CURDATE()** for joined_on. A procedure can mix parameters and functions in the same VALUES list.

**3.** Leave member_id and total_paid out. member_id is a surrogate key the server numbers, and total_paid has a DEFAULT of 0.00 from Project 9.

**4.** A semicolon, then END.

### Step 5. make_payment

```
-- takes a payment. the triggers do the bookkeeping
CREATE PROCEDURE make_payment(IN p_member_id INT,
                              IN p_amount DECIMAL(7,2),
                              IN p_method VARCHAR(10))
BEGIN
    -- store the payment and let the triggers do the rest
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<
END
```

**Writing the statement that goes in the gap**

**1. INSERT INTO payment** naming three columns: member_id, amount and method.

**2. VALUES** with the three parameters in the same order. No quotes around them.

**3.** Leave payment_id out, because the server numbers it from 5000. Leave paid_on out, because the BEFORE INSERT trigger fills in today's date.

**4.** A semicolon, then END.

Notice what you are **not** writing: no UPDATE of total_paid and no INSERT into payment_log. The two Project 9 triggers on payment already do both every time a row arrives, whoever inserts it. Writing them here as well would put two log lines in payment_log for every payment, and an UPDATE that adds the amount to total_paid would count it twice.

## Step 6. refund_payment

Most of this one is yours. There is no skeleton. Lab 5g Step 7 is the model.

| Requirement | Detail |
|---|---|
| Name | refund_payment |
| Parameter | One IN parameter, p_payment_id, INT |
| Body | One DELETE that removes the payment with that payment_id, and only that one |

**Walking through it**

**1.** CREATE PROCEDURE, the name, and the parameter list in brackets.

**2.** BEGIN.

**3. DELETE FROM payment** with a WHERE clause comparing payment_id to the parameter. payment_id is the primary key, so one condition is enough to pick out exactly one row. Compare Lab 5g, where test_scores needed two conditions because no single column identified the score you meant.

**4.** END. Select the whole block and run it.

Read the WHERE twice before you run it. This procedure deletes money from the club's books every time it is called.

Again, no UPDATE and no log entry. The AFTER DELETE trigger from Project 9 handles both.

## Step 7. Change list_members to show the biggest payers first

The treasurer would rather see who has paid the most. Change list_members so the rows are sorted by **total_paid**, highest first, and members with the same total are sorted by **last_name**, A to Z. You write both statements: drop the procedure, then create it again. Lab 5f Step 6 is the model.

The second sort is not decoration. Two members on 0.00 have no order between them unless you give them one, and without it the server is free to list them either way round. ORDER BY takes a list: the second column only decides between rows the first one calls a tie.

Call it. Caleb and Noor are tied on 45.00, so the last name decides:

| member_id | first_name | last_name | total_paid |
|---|---|---|---|
| 102 | Sofia | Marchetti | 60.00 |
| 100 | Noor | Haddad | 45.00 |
| 101 | Caleb | Ortiz | 45.00 |
| 103 | Desmond | Blake | 0.00 |

# Part 3. Use the front door

### Step 8. Run the club for a day

You write every statement in this step. From here on, do everything through your procedures, with one exception: you may select from payment_log at the end to check the triggers did their part.

| | Do this | What you should see |
|---|---|---|
| a | Add a new member, Imani Brooks, then call list_members | Five members. Imani is 104 with 0.00, listed after Desmond because both are on 0.00 and Blake comes before Brooks |
| b | Imani pays 50.00 by card. Then call member_history for her | One row: payment 5004, 50.00, today's date, CARD |
| c | Refund payment 5000, Noor's 45.00 | No rows come back. A DELETE shows nothing |
| d | Call list_members | Sofia 60.00, Imani 50.00, Caleb 45.00, Desmond 0.00, Noor 0.00 |
| e | SELECT * FROM payment_log | Seven rows. Row 6 is Imani's 50.00 ADDED, row 7 is Noor's 45.00 REMOVED |

Look at what you typed in this step: a handful of short CALLs. Behind them the server numbered a member and a payment, dated the payment, capitalized the method, updated two totals and wrote two log lines. That is the front door working.

### Step 9. Three problems, left open on purpose

Your procedures do whatever they are asked, including when the request makes no sense. Run these three and, under each, write a comment saying what happened and what **should** have happened.

| | Do this | What happens |
|---|---|---|
| 1 | A payment of 20.00 in cash for member 999 | ERROR 1452, a foreign key constraint fails. Safe, but not a message a club treasurer would understand |
| 2 | A payment of minus 10.00 in cash for Desmond, member 103. Then list_members | Accepted. Desmond now shows minus 10.00, and the log says ADDED |
| 3 | Refund payment 7777 | Nothing at all. No error, no rows removed. The caller cannot tell a refund that worked from one that matched nothing |

Clean up after problem 2: call member_history for Desmond to find the id of that minus 10.00 payment, and refund it. He goes back to 0.00 and the log gains a REMOVED line, nine rows in all.

> **Why the id is probably 5006 and not 5005**
>
> Imani's payment was 5004, so you might expect the next one to be 5005. On our test server it was **5006**. The failed insert for member 999 in problem 1 took number 5005 from the counter before the foreign key refused the row, and as you saw in Lab 5e, the counter never goes back. Use whatever id member_history shows you, not a number you assume.

> **Where this goes next**
>
> All three problems have the same cure: the procedure should check before it acts, and refuse with a clear message of its own. That is what Lab 5h teaches. Keep this file, because you will be fixing these procedures soon.

### Step 10. List every routine in club_db

Write the routines query yourself, for club_db, in name order. Six rows:

| routine_name | routine_type |
|---|---|
| add_member | PROCEDURE |
| list_members | PROCEDURE |
| make_payment | PROCEDURE |
| member_history | PROCEDURE |
| member_total | FUNCTION |
| refund_payment | PROCEDURE |

## Part 4. Push to GitHub

### Step 11. Publish to a new repository

Save project10_starter.sql first. If you are not signed in to GitHub, click **Accounts** at the bottom of the left bar, choose **Sign in with GitHub**, and click **Authorize**.

1. Click **Source Control**, then **Publish to GitHub**.
2. Name it **cis4720-project10**. It has to be a name you have not used.
3. Choose **Publish to a public repository**. A private one gives your instructor a 404 page instead of your work.
4. Tick **project10_starter.sql**. Leave the **.vscode** folder unticked, because that is the one holding your password.

### Step 12. Check it and submit the URL

1. Open the repository on github.com, click the file and read it on the page. What is there is exactly what gets graded.
2. Open the URL in a private or incognito window. A **404** means it is private, so go to **Settings** and change the visibility to public.
3. Paste the URL into the Project 10 submission box on Canvas.

Submit the URL even if you did not finish. Partial work your instructor can read scores better than finished work nobody can reach.

This is homework, so work on your own. Looking syntax up in your textbook or your Lab 5f and 5g files is expected. Copying another student's file is not.

## How it is graded

| Steps | What is being checked | Points |
|---|---|---|
| 1 | Schema created and the starter file running | 0.5 |
| 2 | list_members written and called | 1.0 |
| 3 | member_history written and called for members 100 and 103 | 1.0 |
| 4 | add_member created, with the INSERT you wrote | 1.0 |
| 5 | make_payment created, with the INSERT you wrote and no extra bookkeeping | 1.0 |
| 6 | refund_payment written, with a DELETE limited to one payment | 1.5 |
| 7 | list_members dropped and recreated with both sort columns | 1.0 |
| 8 | All five parts run, through the procedures | 1.0 |
| 9 | All three problems run with comments, and the cleanup done | 0.5 |
| 10 | The routines query written and run | 0.5 |
| All | Required. A comment in your own words above every statement | 1.0 |
| 11 to 12 | A new public repository with the file in it, URL submitted | 0.0 |

Total 10 points. The due date is listed on Canvas LMS.

## If something goes wrong

| What you see | What it means |
|---|---|
| ERROR 1064 near 'DELIMITER $$' | Delete the DELIMITER lines and the $$ after END. |
| ERROR 1064 near '--something' | A comment lost the space after its two dashes. |
| ERROR 1304, PROCEDURE already exists | Run DROP PROCEDURE IF EXISTS and the name, then create it again. |
| ERROR 1305, PROCEDURE does not exist | It was not created, or it went into another schema. Check SELECT DATABASE(); and the routines query. |
| ERROR 1318, incorrect number of arguments | make_payment takes three values, add_member two, the others one or none. |
| ERROR 1452, cannot add or update a child row | The member does not exist. Members are 100 to 104, not 1 to 5. Expected in Step 9 only. |
| A payment counted twice in total_paid | make_payment has its own UPDATE adding the amount, on top of what the trigger already did. Remove it. |
| Two log lines for one payment | make_payment has its own INSERT into payment_log. Remove it. |
| Every payment disappeared | refund_payment has no WHERE clause. Connect to information_schema, drop club_db, run the starter again, then rerun your own steps. |